Tema 8 — Paralelismo (fork-join, work/span)

Fuentes: teórica 10-paralelismo (fork-join, DAG de cómputo, work/span, scheduler greedy, work stealing, MapReduce/streams), Guía 8 ("Cómputo Paralelo"). No hay apunte de práctica de este tema. **En el parcial entran ejercicios tipo guía, no teoría directa:** cuentas con las cotas y análisis de work/span de algoritmos fork-join.

---

1. La idea

Lanzar un thread por cada pedacito de trabajo (ej. uno por cada `c_ij` de un producto de matrices) es muy paralelo pero caro: crear, planificar y destruir threads de sistema operativo cuesta. La alternativa es un **pool de threads** de larga vida que van tomando **tareas** (unidades cortas de trabajo).

El estilo **fork-join** es divide & conquer con paralelismo: 1. Si el problema es chico (umbral / caso base), lo resolvés secuencialmente. 2. Si no, lo partís en subproblemas, **lanzás** (fork) algunos como tareas, **hacés** (compute) uno vos mismo, y **esperás** (join) a los lanzados. 3. Combinás los resultados.

> **En criollo:** sos jefe de una cuadrilla. Partís la pila de trabajo en dos, una mitad se la dejás en la mesa a quien la agarre (fork), la otra la hacés vos (compute), y cuando terminás esperás a que te traigan la otra (join). Cada uno que agarra una mitad hace lo mismo con la suya.

2. Notación de la cátedra (Java `ForkJoinPool`) ⭐

Cosa	Qué es
`RecursiveTask<T>`	tarea que **devuelve** un valor (ej. Fibonacci, suma de un vector)
`RecursiveAction`	tarea **con efectos**, sin valor de retorno (escribe en una matriz/array de salida: suma de matrices, transponer, mergesort)
`t.fork()`	lanza la tarea `t` para que corra (eventualmente) en el pool
`t.join()`	espera a que `t` termine y devuelve su resultado (el worker puede tomar otra tarea mientras tanto)
`t.compute()` / `compute(t)`	ejecuta la tarea **en el propio worker**, ya mismo

Equivalencias con otros libros (CLRS): `spawn` = `fork`, `sync` = `join`.

Esqueleto (es exactamente la forma que usa la guía en los ejercicios 7 y 8):

```
void resolver(A, lo, hi) {
    if (hi - lo es chico) {
        // caso base secuencial
    } else {
        int mid = (lo + hi) / 2;
        low = new resolver(A, lo, mid);
        low.fork();                          // una mitad, a la cola
        compute(new resolver(A, mid + 1, hi)); // la otra, la hago yo
        low.join();                          // espero la lanzada
        combinar(A, lo, mid, hi);            // trabajo después del join
    }
}
```

Con valor de retorno (Fibonacci de la teórica):

```
class FibTask extends RecursiveTask<Integer> {
    int arg;
    protected Integer compute() {
        if (arg > 1) {
            FibTask right = new FibTask(arg - 1);
            right.fork();
            FibTask left = new FibTask(arg - 2);
            return left.compute() + right.join();   // compute ANTES que join
        } else return arg;
    }
}
```

Con k subtareas (suma de matrices por cuadrantes, de la teórica):

```
List<MatrixAddTask> tasks = ...;   // 4 tareas, una por cuadrante (i,j)
tasks.forEach(t -> t.fork());
tasks.forEach(t -> t.join());      // los join lo más tarde posible
```

Reglas de uso: - **Demorá los `join()` lo más posible.** Primero lanzás todo, después esperás. - **`fork` de una, `compute` de la otra, recién después `join`.** Si hacés `join` antes de `compute`, serializaste todo (ver trampas).

3. El DAG de cómputo

Se modela el cómputo como un **grafo acíclico dirigido**: cada nodo es un **paso**, las aristas son "este paso tiene que ir antes que aquel". - Un paso con `fork()` o `compute()` tiene **dos aristas de salida**: al paso siguiente de la misma tarea y al primer paso de la tarea invocada. - Con `compute()`: hay arista del **último paso de la tarea invocada** al paso siguiente del que hizo el compute. - Con `fork()` de `t'` desde `t`: en `t` hay un paso con `t'.join()`; hay arista del **último paso de `t'`** a ese paso del `join`. - Las aristas siempre van "hacia adelante en el tiempo" → es acíclico.

Camino crítico = el camino más largo desde el paso inicial. Es la cadena de pasos interdependientes que no podés acortar con más procesadores. En Fibonacci(4) de la teórica tiene **8 nodos**.

Formas típicas de DAG (sirve para preguntas tipo "¿qué forma tiene el DAG?"):

Algoritmo	Forma del DAG
D&C con 2 llamadas independientes	árbol binario que se abre (forks) y otro que se cierra (joins): un "rombo"
D&C con k llamadas (cuadrantes)	lo mismo pero k-ario
una sola llamada recursiva (o llamadas que dependen una de la otra)	**una cadena** (camino): no hay nada en paralelo, work ≈ span
recursión desbalanceada (Fibonacci)	árbol torcido; el camino crítico va por la rama más profunda

4. Las métricas ⭐

Símbolo	Nombre	Qué es
`T_P`	latencia	tiempo (en pasos) con P procesadores
T_1	**work** (trabajo)	tiempo con 1 procesador = **total de nodos del DAG**
T_∞	**span** (duración)	tiempo con infinitos procesadores = **largo del camino crítico**
T_1 / T_P	**speedup** (aceleración)	cuánto ganaste con P procesadores. **Lineal** si $T_1/T_P \in \Theta(P)$
T_1 / T_∞	**paralelismo**	el **máximo speedup posible**; "cuántas tareas hay disponibles en promedio". Usar más procesadores que esto no sirve

Las dos leyes (cotas inferiores, valen para cualquier scheduler): - **Ley del work:** $T_P \geq T_1 / P$ (equivalente: $T_1 \leq P \cdot T_P$). No podés repartir mejor que perfecto. - **Ley del span:** $T_P \geq T_\infty$ (equivalente: $T_\infty \leq T_P$ para todo P). No podés romper las dependencias.

Juntas: $T_P \geq \max(T_1/P, T_\infty)$.

> **En criollo:** el work es cuánto trabajo hay en total; el span es la cadena más larga de cosas que dependen una de otra. Con P procesadores tardás al menos lo que te da repartir el trabajo perfecto, y al menos lo que tarda la cadena más larga.

5. Scheduler greedy (la cota superior) ⭐

Greedy: en cada paso de ejecución toma, de los nodos **listos** (con todos sus predecesores ya hechos), hasta P y los ejecuta. Un nodo del DAG = un paso en un procesador.

Teorema (cota greedy):

$$T_P \leq T_1/P + T_\infty$$

Idea de la demo (la necesitás para el Ej 2): - Dividís los pasos de ejecución en **completos** (había ≥ P nodos listos, usás los P procesadores) y **incompletos** (había menos de P, se ejecutan todos los listos). - Completos: $c \leq T_1/P$ (cada uno ejecuta P nodos y en total hay T_1). - Incompletos: $k \leq T_\infty$ (en un paso

incompleto se ejecutan **todos** los listos, incluido el primer nodo pendiente de cada camino crítico del DAG que queda; entonces el span de lo que falta baja en 1). - $T_P = c + k \le T_1/P + T\infty$.

Corolario: greedy es una 2-aproximación del óptimo. Si T_P^* es el óptimo: $T_P^* \ge \max(T_1/P, T\infty)$ y $T_P \le T_1/P + T\infty \le 2 \cdot \max(T_1/P, T\infty) \le 2 \cdot T_P^*$.

Ojo, que sea greedy no quiere decir que sea óptimo: con P = 2 y pasos $A1 \to A2$, B, C, un greedy puede hacer $\{B, C\}, \{A1\}, \{A2\}$ (3 pasos) cuando lo óptimo es $\{A1, B\}, \{A2, C\}$ (2 pasos).

Variante ajustada (Ej 2 de la guía): $T_P \le (T_1 - T\infty)/P + T\infty$ (con P > 1). Notá que es lo mismo que $T_1/P + T\infty \cdot (1 - 1/P)$.

Resumen de todas las cotas:

Tipo	Fórmula	Vale para
inferior	$T_P \ge T_1/P$	cualquier scheduler
inferior	$T_P \ge T\infty$	cualquier scheduler
superior	$T_P \le T_1/P + T\infty$	greedy
superior ajustada	$T_P \le (T_1 - T\infty)/P + T\infty$	greedy, P > 1
estimación	$T_P \approx T_1/P + T\infty$	cuando el enunciado dice "estimar con la cota greedy" (Ej 1)

6. Ejercicios numéricos con las cotas ⭐

Hay dos tipos en la guía:

Tipo A — "estimá T_P con la cota greedy y compará programas" (Ej 1). 1. Escribí $T_P \approx T_1/P + T\infty$ para **cada** programa, con sus T_1 y $T\infty$ (los que no te dan son incógnitas). 2. Traducí cada frase del enunciado en una desigualdad ("con 32 tarda menos" → $T_1'/32 + T\infty' < T_1/32 + T\infty$). 3. Despejá: suele quedar una condición sobre la diferencia de spans ($T\infty' - T\infty$) encerrada entre dos números.

> **En criollo:** con pocos procesadores domina el término T_1/P (gana el que tiene menos work); con muchos, domina $T\infty$ (gana el que tiene menos span). Reducir el work a costa de alargar el span te puede salir caro en la máquina grande.

Tipo B — "midieron T_P para ciertos P, acotá T_P para otro P" (Ej 3). 1. De cada medición sacás **cotas superiores** de T_1 y $T\infty$ usando las leyes: - ley del work → $T_1 \le P \cdot T_P$ (usá la medición que dé la mejor cota); - ley del span → $T\infty \le T_P$ (usá la medición con el T_P más chico). 2. Enchufás esas cotas en la cota greedy (o la ajustada) para el P pedido. Vale enchufar cotas superiores porque la fórmula es **creciente** en T_1 y en $T\infty$ (en la ajustada, $T\infty$ aparece multiplicado por $1 - 1/P > 0$). 3. Si da fraccionario, son pasos: podés redondear hacia abajo una cota superior entera.

7. Calcular work y span de un algoritmo fork-join ⭐⭐

Reglas de composición (lo único que necesitás para armar las recurrencias):

Composición	Work	Span
A **y después** B (secuencial)	`W_A + W_B`	`S_A + S_B`
A **en paralelo con** B (fork/compute + join)	`W_A + W_B`	**`máx(S_A, S_B)`**
k tareas en paralelo (k constante)	suma de los k	máx de los k

Para un D&C con `a` subproblemas de tamaño `n/b`, todos en paralelo, más un trabajo `f(n)` de partir/combinar (con span `g(n)`):

```
W(n) = a · W(n/b) + f(n)
S(n) =     S(n/b) + g(n)      ← una sola copia, porque corren en paralelo
```

- Si el combinar es secuencial (ej. merge lineal), `g(n) = f(n)`.
- Si el combinar está paralelizado, `g(n)` es **su span**, no su work.
- Si las llamadas recursivas **dependen una de otra** (la segunda necesita el resultado de la primera), no son paralelas: el span suma las dos.

Ejemplos de la teórica:

Algoritmo	Work	Span	Paralelismo
suma de matrices n×n (4 cuadrantes)	$A_1(n) = 4A_1(n/2) + \Theta(1) = \Theta(n^2)$	$A_\infty(n) = A_\infty(n/2) + \Theta(1) = \Theta(\log n)$	$\Theta(n^2 / \log n)$
producto de matrices (8 productos + 1 suma)	$M_1(n) = 8M_1(n/2) + A_1(n) = 8M_1(n/2) + \Theta(n^2) = \Theta(n^3)$	$M_\infty(n) = M_\infty(n/2) + A_\infty(n) = M_\infty(n/2) + \Theta(\log n) = \Theta(\log^2 n)$	$\Theta(n^3 / \log^2 n)$ (con n = 1000, ~10^7 procesadores)
Fibonacci(n)	exponencial (como la recursión secuencial)	$\Theta(n)$ (la rama `n-1, n-2, …`)	exponencial / n

Fijate en el producto: **el combinar (la suma de matrices) entra al span con su span $\Theta(\log n)$, no con su work $\Theta(n^2)$**, porque la suma también está paralelizada.

8. Tabla de recurrencias resueltas ⭐⭐

(Todas con caso base $\Theta(1)$. Las de la forma $T(n/2) + \dots$ son las de **span**; las de $2T(n/2) + \dots$ o $4T(n/2) + \dots$, de **work**.)

Recurrencia	Solución	Dónde aparece
$T(n) = 2T(n/2) + \Theta(1)$	$\Theta(n)$	work de sumar un vector por D&C
$T(n) = 2T(n/2) + \Theta(\log n)$	$\Theta(n)$	work de algo que parte en 2 con costo log (ej. merge paralelo)
$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$	work de mergesort
$T(n) = 2T(n/2) + \Theta(n \log n)$	$\Theta(n \log^2 n)$	
$T(n) = 4T(n/2) + \Theta(1)$	$\Theta(n^2)$	work de suma de matrices / recorrer una matriz por cuadrantes
$T(n) = 4T(n/2) + \Theta(n^2)$	$\Theta(n^2 \log n)$	
$T(n) = 8T(n/2) + \Theta(n^2)$	$\Theta(n^3)$	work de producto de matrices
$T(n) = T(n/2) + \Theta(1)$	$\Theta(\log n)$	span de D&C con combinar constante
$T(n) = T(n/2) + \Theta(\log n)$	$\Theta(\log^2 n)$	span de producto de matrices; span de un combinar de costo log
$T(n) = T(n/2) + \Theta(\log^2 n)$	$\Theta(\log^3 n)$	span de D&C cuyo combinar tiene span $\log^2 n$
$T(n) = T(n/2) + \Theta(n)$	$\Theta(n)$	span de D&C con combinar **lineal secuencial**
$T(n) = T(3n/4) + \Theta(\log n)$	$\Theta(\log^2 n)$	span si la partición deja a lo sumo 3/4 de un lado
$T(n) = T(n-1) + \Theta(1)$	$\Theta(n)$	cadena lineal (recursión que achica de a 1)
$T(n) = T(n-1) + T(n-2) + \Theta(1)$	$\Theta(\phi^n)$	work de Fibonacci

Cómo sacarlas sin memorizar: - **Span $S(n) = S(n/b) + g(n)$:** hay $\log_b n$ niveles y sumás g en cada uno. Si g es polilog $\log^k n$ → $\Theta(\log^{k+1} n)$. Si g es polinomial (n, n^2) → manda el primer término: $\Theta(g(n))$ (es una serie geométrica). - **Work $W(n) = a \cdot W(n/b) + f(n)$** (teorema maestro): compará $f(n)$ con $n^{\log_b a}$ (cantidad de hojas). - f más chica polinomialmente (ej. $\Theta(1)$, $\Theta(\log n)$ contra n) → $\Theta(n^{\log_b a})$, mandan las hojas. - $f = \Theta(n^{\log_b a} \cdot \log^k n)$ → $\Theta(n^{\log_b a} \cdot \log^{k+1} n)$, todos los niveles pesan igual. - f más grande polinomialmente → $\Theta(f(n))$, manda la raíz. - Si te piden "como recurrencia y en Θ", escribí las dos cosas: primero la recurrencia con su caso base, después la solución.

9. Receta para "escribí / analizá este algoritmo fork-join" ⭐

1. **Pensá el D&C secuencial:** ¿cómo parto? (mitades del índice, cuadrantes de la matriz, exponente a la mitad, la mediana...). ¿Cuál es el caso base?
2. **¿Las llamadas recursivas son independientes?** Que no escriban en las mismas posiciones y que ninguna lea lo que otra escribe. Si lo son → `fork` de todas menos una, `compute` de la última, `join`. Si una necesita el resultado de la otra → **no hay paralelismo** ahí.
3. **Elegí el tipo de tarea:** devuelve valor → `RecursiveTask<T>` (combinás con `compute() + join()`); escribe en una salida → `RecursiveAction`.
4. **Escribí el pseudocódigo** con la notación de la sección 2 (`low.fork(); compute(...); low.join();`).
5. **Work:** $W(n) = (\text{cantidad de llamadas}) \cdot W(n/b) + (\text{costo de partir + combinar})$.

6. **Span:** `S(n) = S(n/b) + (span de partir + combinar)`. Una sola copia de la recursión si son paralelas.
7. **Resolvé** con la tabla de la sección 8. No te olvides de los casos base.
8. **Paralelismo** = `W(n) / S(n)`. Simplificá (ej. `Θ(n log n) / Θ(n) = Θ(log n)`).
9. **Si te preguntan si es "bueno":** paralelismo chico (`Θ(1)`, `Θ(log n)`) → el cuello de botella está en algún paso secuencial del span; buscá cuál (casi siempre el combinar) y paralelizalo.
10. **Si te piden justificar correctitud / ausencia de races** (Ej 8a): mostrá que las tareas paralelas **escriben en regiones disjuntas** de la salida, y que lo que leen **nadie lo modifica** mientras corren. Después, que cada elemento termina en la posición correcta (cuántos elementos son menores que él).

10. Work stealing y otras cosas de la teórica (no entran como ejercicio)

- **ThreadPoolExecutor:** una única cola de tareas → contención. **ForkJoinPool:** una cola por worker + work stealing.
- **Work stealing:** cada worker tiene una **deque**. El dueño hace `pushBottom()` (cada `fork`) y `popBottom()` (LIFO: tareas nuevas, calientes en caché). Un worker sin tareas se vuelve **ladrón**: elige víctima al azar, hace `Thread.yield()` y le roba la tarea **más vieja** con `popTop()`. Cada uno en un extremo → poca contención; el top se implementa lock-free con CAS (y `popBottom` usa CAS cuando queda un solo elemento). `popTop()` puede devolver `null` si pierde contra otro ladrón: reintenta con otra víctima.
- **Work dealing:** en vez de robar, cada thread rebalancea con probabilidad `1/(s+1)` (s = tamaño de su deque) con un socio al azar, tomando los dos locks en orden de ID (sin deadlock), y transfiere tareas si la diferencia supera un umbral.
- **MapReduce:** fragmentos → mappers (pares clave/valor) en paralelo → grouping por clave → reducers en paralelo → mapa de salida. Mappers y reducers son tareas del pool fork-join. Ejemplos: WordCount, KMeans.
- **Streams:** pipelines lazy, funcionales. `parallelStream()` usa ForkJoinPool por debajo. Las lambdas no tienen que ser **interferentes** (modificar la fuente) ni **stateful** (ej. `target.add(s)` en paralelo → race); y la reducción tiene que ser asociativa.

Ley de Amdahl (no está en la teórica ni en la guía, pero por si aparece): si una fracción `p` del programa se paraleliza perfectamente y `1 − p` es secuencial, `speedup ≤ 1 / ((1 − p) + p/P)`, y con P → ∞ el speedup tope es `1/(1 − p)`. Es la misma idea que "el paralelismo `T₁/T∞` es el speedup máximo": la parte secuencial juega el papel del span.

11. Trampas ⭐

1. **`join` antes de `compute` = secuencial.** `low.fork(); low.join(); compute(high);` espera a que termine la primera antes de arrancar la segunda: el span pasa a ser `S_low + S_high`, no el máximo. Siempre: fork, compute, join.
2. **Hacer `fork` de todas y `join` de todas sin `compute`** no está mal, pero desperdicia al worker actual. En la notación de la guía: fork de una, compute de la otra.
3. **Poner el work del combinar en el span cuando el combinar es paralelo.** En el span va el **span** del combinar (producto de matrices: `+ A∞(n) = Θ(log n)`, no `+ Θ(n²)`).
4. **Contar dos veces la recursión en el span.** Si las dos mitades son paralelas, `S(n) = S(n/2) + …`, no `2S(n/2) + …`.
5. **Inventar paralelismo duplicando trabajo.** Si para paralelizar calculás dos veces lo mismo (ej. dos llamadas idénticas en vez de una y reusar el resultado), el span no mejora y el work explota. Si las

llamadas son iguales, se hace una sola y se reusa.

6. **Llamadas que dependen entre sí no se pueden forkear juntas.** Si la segunda usa el resultado de la primera, el DAG es una cadena.
7. **Leyes vs cota greedy:** las leyes ($T_P \geq T_1/P$, $T_P \geq T_\infty$) son **inferiores** y valen siempre; la cota $T_1/P + T_\infty$ es **superior** y solo para greedy. En el Ej 3 se usan las leyes "al revés" para acotar T_1 y T_∞ por arriba.
8. **El sentido de las desigualdades al enchufar cotas:** solo podés reemplazar T_1 y T_∞ por cotas **superiores** en una cota superior (porque es creciente en ambos).
9. **Speedup ≠ paralelismo.** Speedup es T_1/T_P para un P concreto; paralelismo es T_1/T_∞, el techo del speedup.
10. **Forkear k tareas en un `for`** cuesta $\Theta(k)$ pasos de lanzamiento en el span. Con k constante (4 cuadrantes, 8 productos) es $\Theta(1)$; con k = n, no.
11. **Races en `RecursiveAction`:** si dos tareas paralelas escriben en el mismo lugar de la salida, hay condición de carrera. Las particiones tienen que ser disjuntas.

12. Patrones para la Guía 8

Ej	Técnica (sin resolver)
1	Tipo A (sección 6): estimar $T_P \approx T_1/P + T_\infty$ para Π y Π' (works dados, spans incógnitas), plantear las dos desigualdades (P = 32 y P = 512) y despejar una condición sobre $T_\infty' - T_\infty$. Es la historia de los programas de ajedrez del Cormen.
2	Demo de la cota ajustada (sección 5): misma partición en c completos + k incompletos; acotá c mejor usando que cada paso incompleto también ejecuta al menos un nodo, y después usá $k \leq T_\infty$ (ahí entra $P > 1$).
3	Tipo B (sección 6): leyes del work y del span sobre T_4 y T_{64} para acotar T_1 y T_∞ por arriba, y enchufar en la cota del Ej 2 con P = 10.
4	D&C sobre el vector en dos mitades con `RecursiveTask<Integer>` (devuelve la suma), `fork/compute/join` y sumar. Recurrencias de la fila $2T(n/2) + \Theta(1)$ y $T(n/2) + \Theta(1)$.
5	Potenciación por cuadrados: mirá cuántas llamadas recursivas hay realmente y si dependen entre sí → forma del DAG (sección 3). Ojo con la trampa 5. Recursión sobre el exponente `c` , no sobre `X` .
6	Igual que la **suma de matrices** de la teórica: 4 cuadrantes como `RecursiveAction` , pero el cuadrante (i, j) de A va al cuadrante (j, i) de A_T. Recurrencias con $4T(n/2) + \Theta(1)$.
7	Mergesort con merge **secuencial**: work con $2T(n/2) + \Theta(n)$, span con $T(n/2) + \Theta(n)$ (el merge entra entero al span). Paralelismo chico → motiva el Ej 8.
8	Merge paralelo partiendo por la mediana. (a) regiones de escritura disjuntas en A (`pos_en_A` separa izquierda y derecha), T solo se lee; correctitud contando cuántos elementos son menores que la mediana. (b) la mediana parte exactamente a la mitad: work con $2T(n/2) + \Theta(\log n)$, span con $T(n/2) + \Theta(\log n)$. (c) Ej 7 pero reemplazando el $\Theta(n)$ del span por el span del merge nuevo (fila $T(n/2) + \Theta(\log^2 n)$).